

1 · How user code uses tonic

tonic · h2 · tokio · DSA/IAx

The public surface is generated client/server glue plus `Request<T> / Response<T>`.

C — Client: generated method

rust

```
let mut client = GreeterClient::connect(dst).await?;
```

```
let response = client
    .say_hello(Request::new>HelloRequest { name })))
    .await?;
```

```
let reply: HelloReply = response.into_inner();
```

The generated method hides the transport, codec, and HTTP/2 body machinery.

S — Server: generated service

rust

```
Server::builder()
    .add_service(GreeterServer::new(MyGreeter))
    .serve(addr)
    .await?;
```

```
impl Greeter for MyGreeter {
    async fn say_hello(&self, request: Request<HelloRequest>)
        -> Result<Response<HelloReply>, Status>
    { /* user handler */ }
}
```

The handler sees typed Rust values, not frames or DATA chunks.

What stays below the generated API transport · codec · HTTP/2 body machinery · frames/DATA chunks

2 · Down the tonic/prost call stack

tonic · h2 · tokio · DSA/IAX

The call stack is tonic/prost code. The Grpc names below are tonic runtime types, not a separate gRPC library.

C — Client request path

```
GreeterClient::say_hello
└ tonic::client::Grpc::unary
  └ Grpc::streaming
    └ EncodeBody::new_client
      └ Body::poll_frame
        └ EncodeBody::poll_frame
          └ ProstEncoder::encode
            └ prost::Message::encode
```

S — Server request path

```
generated Service::call
└ tonic::server::Grpc::unary
  └ Streaming::new_request
    └ Streaming::message().await
      └ StreamingInner::poll_next
        └ poll_decode_chunk
          └ ProstDecoder::decode
            └ user handler
```

Response mirrors the same shape server::Grpc::map_response builds EncodeBody::new_server; the client returns through Grpc::create_response and Streaming::message().await.

3 · What tonic does underneath for gRPC

tonic · h2 · tokio · DSA/IAx

This is protocol work implemented inside tonic, below the user-facing API and around prost.

gRPC message frame · uncompressed



Protocol work inside tonic

Lazy body polling — bytes appear only when hyper/h2 polls tonic bodies.

Protobuf payload — prost serializes/deserializes typed messages.

gRPC frame — tonic writes/parses [flag:1][len:4][payload].

Compression boundary — optional compression/decompression runs over complete protobuf bytes.

HTTP/2 integration — DATA carries bytes; trailers carry final status/metadata.

Async caveat The logical stack crosses poll points: encode/decode work runs when the body is polled, not necessarily when the user first calls the generated method.

Send path: typed Rust becomes DATA only at the edge

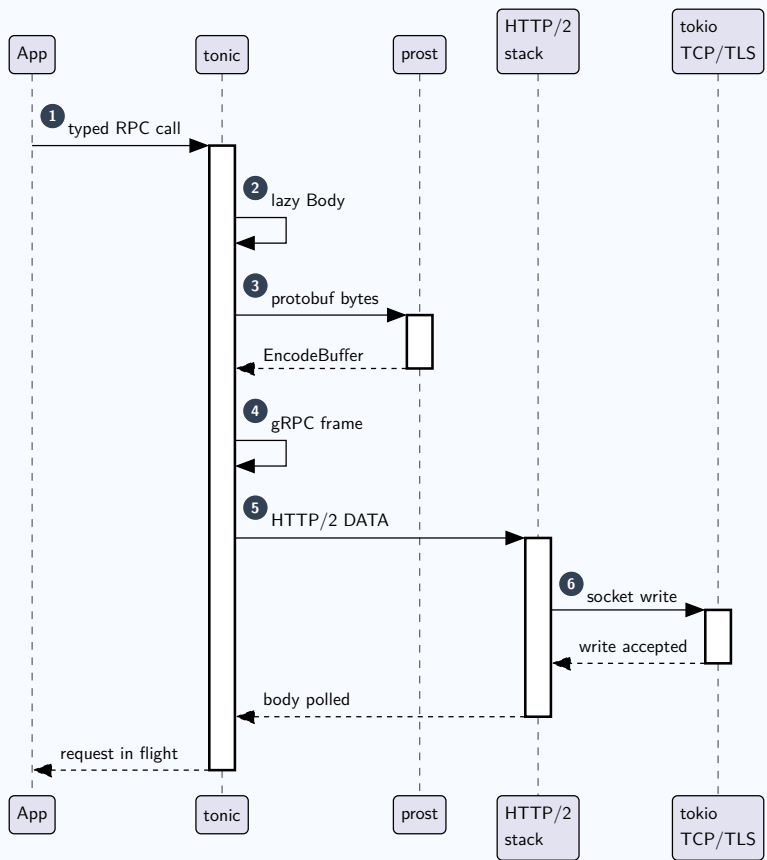
tonic · h2 · tokio · DSA/IAx

component lifeline active call/future span return/data path

Chronos notation: software lifelines, not Rust or packet lifetimes.

overview trace

typed → protobuf → gRPC frame → DATA



Numbers map to the graph

- typed RPC call** — Generated client code starts from a Rust request value.
- lazy Body** — tonic stores request state; bytes appear only when the body is polled.
- protobuf bytes** — prost serializes fields into payload bytes; schema walk stays CPU.
- gRPC framing** — Tonic's codec adds [flag:1][len:4] and may compress the complete payload.
- HTTP/2 DATA** — Tower/Hyper/h2 frame body bytes as DATA.
- socket write** — Tokio/TLS/TCP moves bytes; protobuf semantics are gone.

Overview cut

Tower/hyper/h2/tokio detail is collapsed so the representation changes stay visible; codec and async-poll mechanics reopen on later slides.

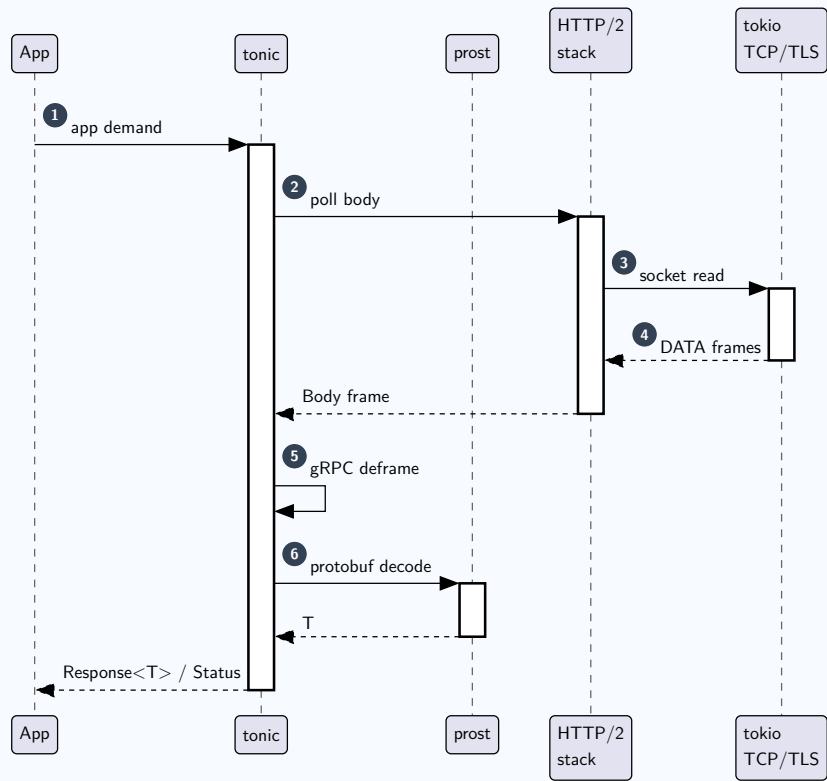
Receive path: DATA becomes Rust only after deframe and decode

| component lifeline | active call/future span ----> return/data path

Chronos notation: software lifelines, not Rust or packet lifetimes.

reverse trace

DATA → gRPC message → protobuf → Rust T



Numbers map to the graph

- 1. app demand** — The app asks tonic for the next response message.
- 2. poll body** — tonic polls the HTTP body when no complete message is buffered.
- 3. socket read** — The lower stack may need Tokio/TLS/TCP bytes before a frame can return.
- 4. DATA frames** — HTTP/2 DATA and trailers return upward; flow control stays lower.
- 5. gRPC deframe** — tonic parses the 5-byte prefix and waits for the announced payload.
- 6. protobuf decode** — Optional decompression happens first; then prost rebuilds typed T.

Reverse shape

DATA bytes become one gRPC-framed message, then protobuf payload bytes, then a typed Rust value or a separate status path.

Compression first; copies only when large; control stays CPU

tonic · h2 · tokio · DSA/IAX

01 · compression first
IAX/IAA is the strongest fit
Use it only where tonic already runs gRPC compression/decompression over complete protobuf bytes, and only when the configured gRPC encoding matches hardware constraints.

02 · copies only when large
DSA is workload dependent
Try data_move or copy_crc only for large unavoidable contiguous copies: large bytes/string append, body assembly, or copy+CRC.

03 · control stays CPU
Do not offload semantics
prost varints/schema traversal, Tower policy, HTTP/2 flow control, and Tokio scheduling are branchy control paths, not bulk transforms.

Offload surface by stage

ordered by fit, not blanket acceleration

Stage	Candidate	Why	Caveat
compress / decompress	IAX/IAA	bulk byte transform at explicit tonic boundary	must match gRPC compression format and IAX/IAA Deflate limits
large bytes/string append	DSA	payload copy can be contiguous	large payload only; tags/varints/schema walk stay CPU
body assembly or copy/CRC	DSA	bulk memory move or copy_crc	only if copy remains unavoidable and completion cost is amortized
prost tags/varints/decode	CPU	schema-dependent branches	keep schema semantics on CPU
h2/Tokio/Tower control	CPU	state machines, flow control, scheduling	measure for attribution, not offload

Evidence grounding: repo architecture notes rank compression/decompression highest and DSA copy/CRC conditional; current Tonic+DSA notes only support trying DSA around 1 MiB+ payloads, with 2–4 MiB more promising but still directional.

Send detail A: tonic encode decision points

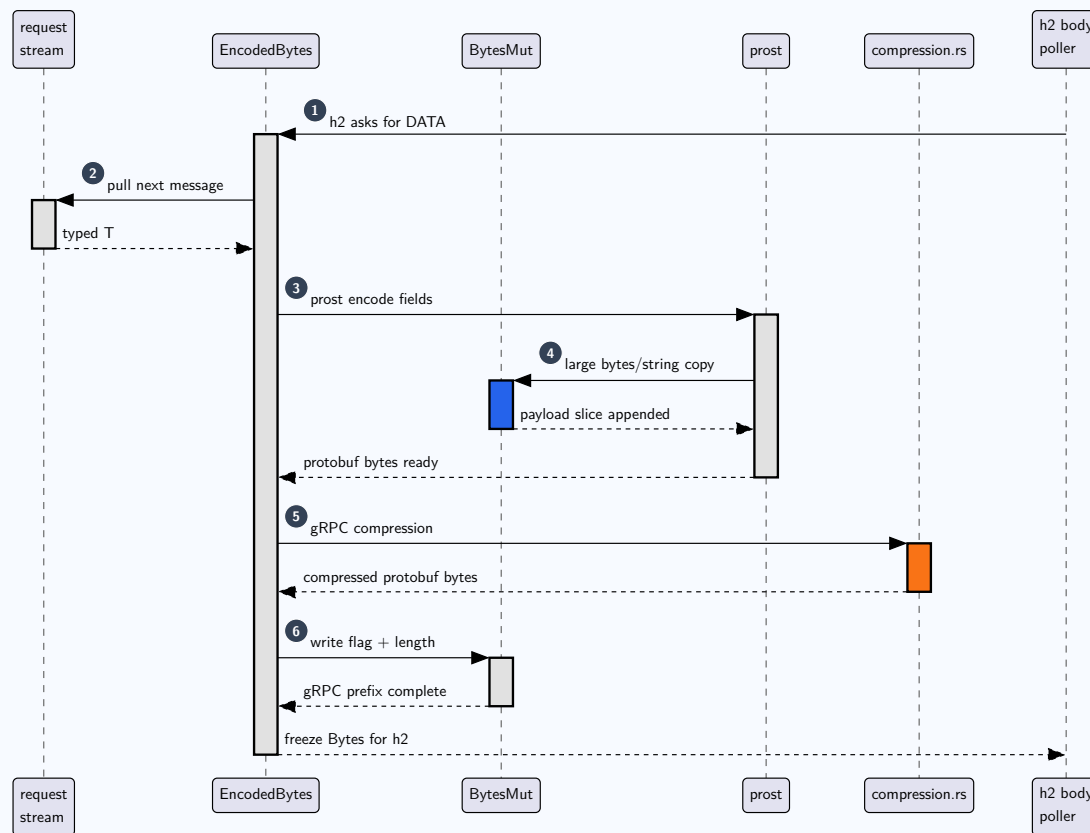
tonic · h2 · tokio · DSA/IAX

orange = IAX compression

blue = DSA copy/CRC

gray = CPU control

candidate spans, not participants



Decision sequence

- 1. Downstream pull** — h2 polls tonic for DATA.
- 2. Message pull** — Tonic pulls one Rust message.
- 3. Protobuf** — prost walks schema and writes fields on CPU.
- 4. Large field copy** — DSA only for big bytes/string appends.
- 5. Compression** — gRPC compression runs after protobuf bytes exist; IAX/IAA candidate.
- 6. Write prefix** — The flag/length prefix and h2 handoff stay CPU.

Boundary rule

Candidate colors stay inside software lifelines. DSA copy/CRC only matters if a large post-encode copy remains; hardware is not a participant.

Async encode: two polls, one owned operation

tonic · h2 · tokio · DSA/IAX

orange = IAX compression

blue = DSA copy/CRC

gray = CPU control

candidate spans, not participants

First poll

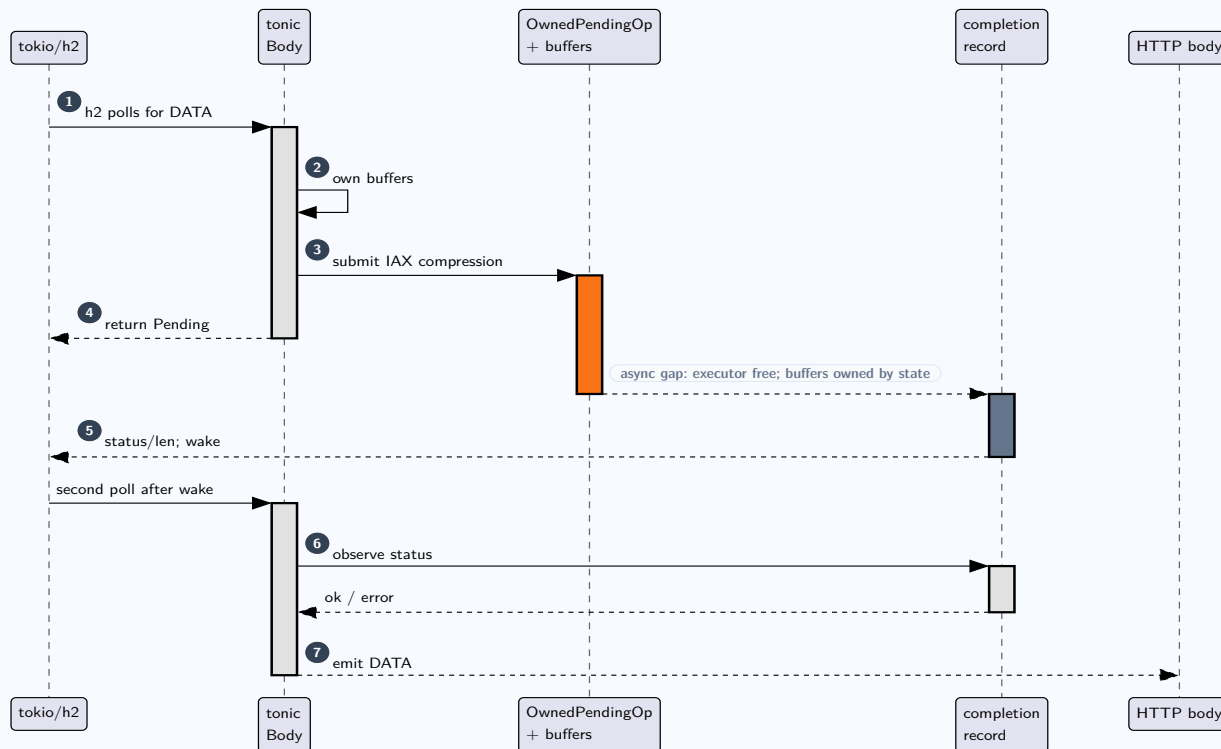
Own buffers, submit IAX compression, then yield `Poll::Pending`.

In flight / Pending

Executor is free; `OwnedPendingOp` + completion record own the operation.

Second poll

Observe status/length; emit DATA on success or surface the error.



Owned async state invariant

- Own src/dst buffers across `Poll::Pending`.
- Completion record stores status/length and wakes the body task.
- Second poll observes success/error; success emits DATA.

DSA caveat

DSA `copy_crc` uses this same two-poll shape for large copies; it is not another numbered flow or hardware participant.

Receive detail A: frame accumulation and deframe

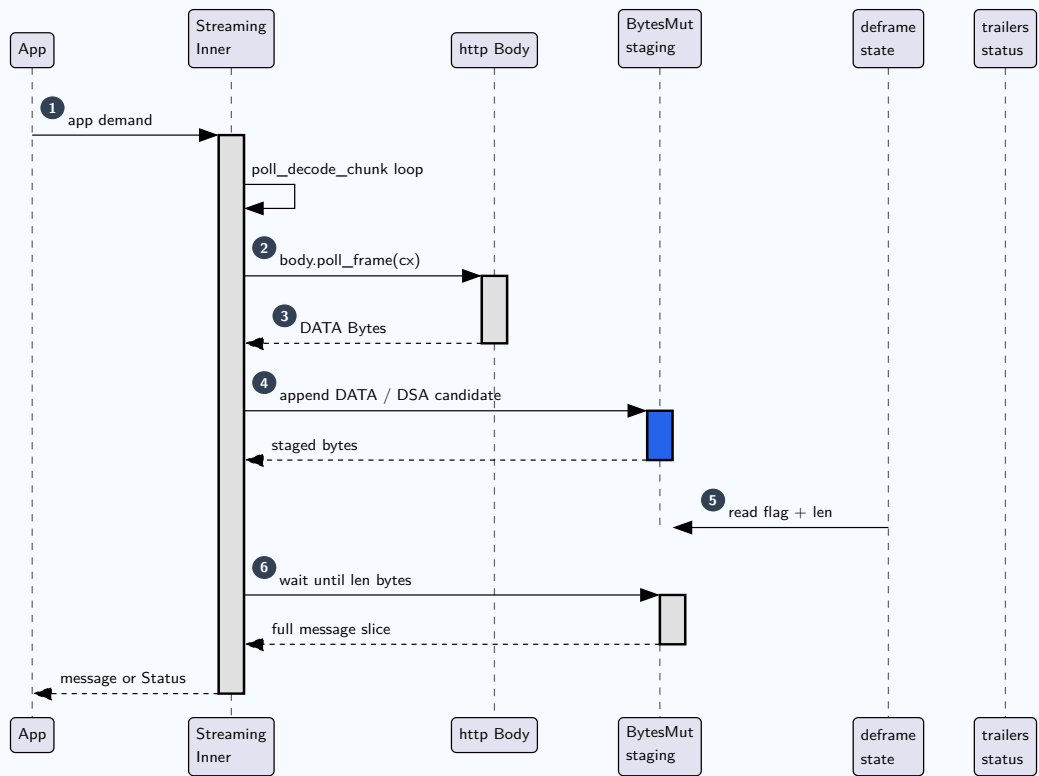
tonic · h2 · tokio · DSA/IAX

orange = IAX compression

blue = DSA copy/CRC

gray = CPU control

candidate spans, not participants



Deframe sequence

- 1. Demand** — The app poll starts receive work.
- 2. Poll body** — No frame means Pending, not a spin loop.
- 3. DATA bytes** — HTTP/2 DATA arrives from the body.
- 4. Append bytes** — DSA only if fragmented assembly becomes a large copy.
- 5. Read prefix** — Tonic parses the 5-byte flag/length on CPU.
- 6. Wait body** — Wait for len bytes; trailers/status stay metadata.

Boundary rule

Candidate colors stay inside software lifelines. DSA/IAX are not deframe participants, and trailers/status stay CPU metadata.

Receive detail B: decompress and typed decode

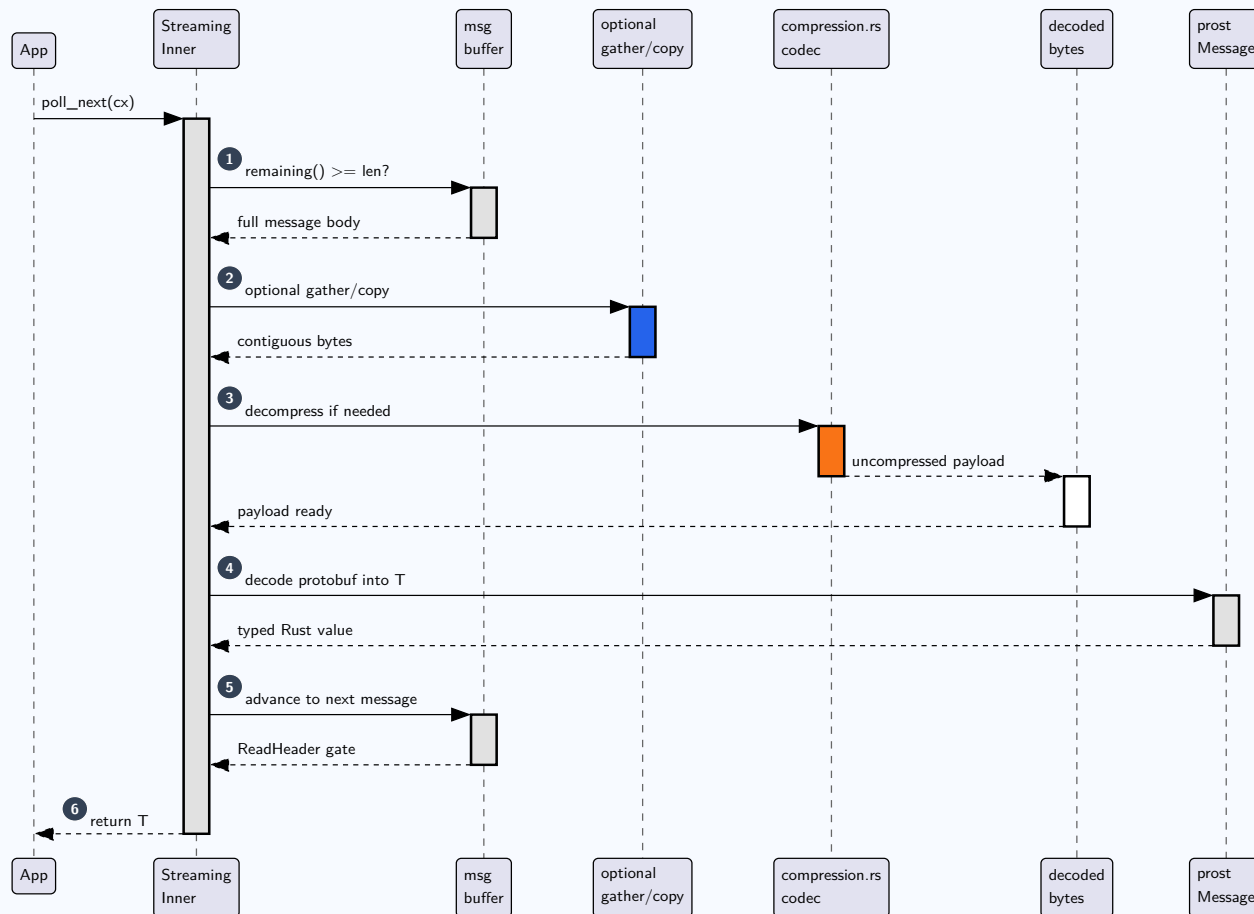
tonic · h2 · tokio · DSA/IAX

orange = IAX compression

blue = DSA copy/CRC

gray = CPU control

candidate spans, not participants



Decode sequence

- 1. Full message** — A complete gRPC message body is buffered.
- 2. Gather/copy** — DSA only for large fragmented payloads.
- 3. Decompress** — Offloaded decompression can yield Pending and wake later.
- 4. Decode** — prost rebuilds Rust T on CPU.
- 5. Advance** — The buffer moves to the next message gate.
- 6. Return** — The stream yields the typed message to the app.

Scope note

Colored spans stay inside software lifelines. Offloaded copy/decompress may yield Pending and wake later; benchmark batch_n separately from concurrency.